

Tessera

written by AI, hardened against AI

Tessera user manual

Writing Tessera by hand · Tessera 0.2

Stefano Stegani - Airgrid Network - www.airgrid.it

Apache 2.0 · DOI 10.5281/zenodo.23855820 · github.com/stegani/tessera

This manual is for people who want to write Tessera **by hand**, without an AI. It describes the language as the 0.2 prototype actually implements it: every complete program in these pages is checked and run by the project's tests. The ideas behind the language, and the parts that are planned but not built yet, are in the paper.

Contents

1. Install and first program
2. Files, modules and names
3. Functions
4. Values and types
5. Variables
6. Control flow and logic
7. Text
8. Lists
9. Your own types
10. match
11. Errors as values: Option and Result
12. Contracts and examples
13. Effects and processes
14. Ownership: edit and take
15. Secret and Untrusted
16. Compiling to WebAssembly
17. Error messages
18. Not available yet
19. Coming from another language

1. Install and first program

You need Python 3.12 or later. From the project folder:

```
python -m venv .venv
```

```
.venv/Scripts/pip install -r requirements.txt
```

On Linux and macOS the path is `.venv/bin/pip`. Then activate the environment and save this as `hello.tsr`:

```
module hello

process main
  effects: console
do
  print("Hello, Tessera!")
end process main
```

```
python -m tessera run hello.tsr
```

Printing is an *effect*, so the program has to say so: `effects: console`. Section 13 explains why.

The commands:

Command	What it does
`python -m tessera run file.tsr`	checks the program, then runs its `process` or its `fn main()`
`python -m tessera check file.tsr`	static checks only, without running (`--json` for one JSON error per line)
`python -m tessera test file.tsr`	runs every `example:` clause as a test
`python -m tessera build file.tsr -o build`	compiles to WebAssembly (`.wat` and `.wasm`)
`python -m tessera call file.tsr name arg...`	calls one function with the given arguments (`--wasm` for the compiled version)

The commands also have Italian names (`esegui` , `verifica` , `prova` , `compila` , `chiama`).

2. Files, modules and names

- A file is a module and starts with `module name` (dotted names are allowed: `module shop.prices`).
- One statement per line, no `;` . Inside brackets you may break lines freely. Indentation does not matter to the parser, but two spaces is the convention.
- Comments start with `--` and run to the end of the line.
- **Type names are in `PascalCase` , every other name is in `snake_case` .** The grammar enforces it: `parse_price` , `ParseError` , `max_items` .
- Code is ASCII only; strings may contain any character.
- `pub` makes a function, type or constant visible to other modules. To use it elsewhere, import it by name (never `*`):

```
from shop.prices import Cents, parse_price, total
```

Imported modules are looked up among the `.tsr` files in the same folder and its subfolders, by their `module` name. examples/checkout.tsr imports from examples/prices.tsr.

Constants are declared at module level, with their type:

```
const max_items: i64 = 100
```

3. Functions

```
module functions

pub fn area(width: i64, height: i64) -> i64
  doc: "Area of a rectangle."
  effects: none
  requires: width >= 0 and height >= 0
  ensures: result >= 0
  example: area(3, 4) == 12
do
  return width * height
end fn area
```

The signature is everything a caller needs, always in the same place and in a **fixed order**:

Clause	Required	Meaning
`doc: "..."`	no	documentation
`effects: none` or `effects: console, fs`	**yes**	what the function may do besides computing (section 13)
`requires: condition`	no, repeatable	what the caller must guarantee; checked on entry
`ensures: condition`	no, repeatable	what the function guarantees; checked on exit, `result` is the returned value
`example: expression`	no, repeatable	a test that must be true; run with `tessera test`

Then `do`, the body, and `end fn` followed by the function's name. Other rules:

- `return` is always explicit, and every path that returns a value must reach a `return`.
- A function without `-> Type` returns nothing.
- Parameters are read-only; section 14 shows how to change the caller's variable.
- There are no classes or methods to declare: `x.f(a)` is another way to write `f(x, a)`.
- Anonymous functions are written `fn(x) => x * 2` and are used with `map` and `filter`.

4. Values and types

Type	Values	Notes
`i64`	`42`, `-7`, `1_000_000`	64-bit integers; overflow **stops the program**, it never wraps around
`i32`, `i16`, `i8`, `u64`, `u32`, `u16`, `u8`		checked where the type is declared (parameters, return values, typed variables)
`f64`	`2.5`, `1.0`	write the decimal point: `2.0`, not `2`
`bool`	`true`, `false`	
`Str`	`"text"`	section 7
`List[T]`	`[1, 2, 3]`	section 8
`(A, B)`	`(1, "a")`	tuples
`Option[T]`	`Some(x)`, `None`	a value or nothing (section 11)
`Result[T, E]`	`Ok(x)`, `Err(e)`	a value or an error (section 11)

Arithmetic rules:

- **Integers and decimals never mix**: `1 + 2.0` is an error. Convert with `n.to_f64()` or `x.to_int()`.
- Integer `/` truncates toward zero and `%` keeps the sign of the dividend: `-7 / 2` is `-3`, `-7 % 2` is `-1`.
- Division by zero stops the program, for decimals too.
- Quantities have units: `512MB`, `4GB`, `30s`, `200ms` (`1KB` is 1024 bytes).

Number methods: `to_f64()`, `to_str()`, `abs()` on integers; `to_int()`, `to_str()`, `abs()`, `floor()`, `round()`, `sqrt()` on decimals. Built-in functions: `min(a, b)`, `max(a, b)`, `abs(x)`, `from_code(n)` (the character with code `n`).

5. Variables

```
module variables

fn sum_to(n: i64) -> i64
  effects: none
  example: sum_to(4) == 10
do
  let start = 1
  var total = 0
  for i in start..n + 1 do
    total += i
  end for
  return total
end fn sum_to

fn evens(limit: i64) -> List[i64]
  effects: none
  example: evens(7) == [0, 2, 4, 6]
do
  var out: List[i64] = []
  for i in 0..limit do
    if i % 2 == 0 do
      out.push(i)
    end if
  end for
  return out
end fn evens
```

- `let` declares a value that never changes; `var` one that can change.
- Assignment: `=`, `+=`, `-=`, `*=`, `/=`. There is no `++`.
- An empty list needs its type: `var out: List[i64] = []`.
- **Values, not references**: `let b = a` makes an independent copy. Changing `a` afterwards does not change `b`.
- `let (x, y) = pair` takes a tuple apart.

6. Control flow and logic

```
module control

fn classify(n: i64) -> Str
  effects: none
  example: classify(-3) == "negative"
  example: classify(0) == "zero"
  example: classify(12) == "large"
do
  if n < 0 do
    return "negative"
  else if n == 0 do
    return "zero"
  else if n > 10 do
    return "large"
  else
    return "small"
  end if
end fn classify

fn first_multiple(values: List[i64], k: i64) -> i64
  effects: none
  requires: k != 0
  example: first_multiple([3, 5, 10, 12], 5) == 5
  example: first_multiple([1, 2], 7) == -1
do
  var found = -1
  for v in values do
    if v % k != 0 do
      continue
    end if
    found = v
    break
  end for
  return found
end fn first_multiple

fn countdown(n: i64) -> List[i64]
  effects: none
  example: countdown(3) == [3, 2, 1]
do
  var out: List[i64] = []
  var i = n
  while i > 0 do
```

```
    out.push(i)
    i -= 1
  end while
  return out
end fn countdown

fn in_range(x: i64, low: i64, high: i64) -> bool
  effects: none
  example: in_range(5, 1, 10)
  example: not in_range(10, 1, 10)
do
  return low <= x and x < high
end fn in_range
```

- Every block ends with `end` plus its keyword: `end if`, `end for`, `end while`, `end match`.
- `for i in 0..n` counts from 0 to n-1: **ranges exclude the end**. For 1 to n write `1..n + 1`.
- `for x in list` goes through a list; `for c in text.chars()` through the characters of a text.
- Logic is written in words: `and`, `or`, `not`, `implies`.
- `and` **and** `or` **never mix without brackets**: write `(a and b) or c`.
- **Comparisons do not chain**: write `low <= x and x < high`, never `low <= x < high`.
- `x in [1, 2, 3]` tests membership.

7. Text

```
module text

fn greet(name: Str, count: i64) -> Str
  effects: none
  example: greet("Ada", 2) == "Hello Ada, you have 2 messages"
do
  return "Hello {name}, you have {count} messages"
end fn greet

fn initials(full_name: Str) -> Str
  effects: none
  example: initials("ada lovelace") == "AL"
do
  var out = ""
  for word in full_name.split(" ") do
    if not word.is_empty() do
      out = "{out}{word.slice(0, 1).upper()}"
    end if
  end for
  return out
end fn initials
```

- Strings use double quotes. **Interpolation is always on**: `"{name}"`, `"{a + b}"`. To write a brace, escape it: `\{` and `\}`.
- Other escapes: `\"`, `\\`, `\n`, `\t`.
- There is no `+` between strings: join them with interpolation (`"{a}{b}"`) or with `parts.join("")`.
- A character is a one-letter `Str`: `"c"`, never `'c'`.

Method	Result
`len()`, `is_empty()`	length, empty or not
`upper()`, `lower()`, `trim()`	changed copy
`contains(s)`, `starts_with(s)`, `ends_with(s)`	`bool`
`replace(a, b)`	copy with every `a` replaced
`split(sep)`	`List[Str]`
`split_once(sep)`	`(Str, Str)`: before and after the first `sep` (after is `""` if missing)
`chars()`	`List[Str]`, one per character
`slice(start, end)`	part of the text; out of range stops the program
`code()`	the code of a one-character text; `from_code(n)` is the reverse
`is_digits()`	non-empty and only ASCII digits 0-9
`to_int()`	`Result[i64, Str]`
`to_float()`	`Result[f64, Str]`

8. Lists

```
module lists

fn positives(values: List[i64]) -> List[i64]
  effects: none
  example: positives([3, -1, 0, 7]) == [3, 7]
do
  return values.filter(fn(v) => v > 0)
end fn positives

fn squares(values: List[i64]) -> List[i64]
  effects: none
  example: squares([1, 2, 3]) == [1, 4, 9]
do
  return values.map(fn(v) => v * v)
end fn squares

fn total(values: List[i64]) -> i64
  effects: none
  example: total([]) == 0
  example: total([4, 5]) == 9
do
  if values.is_empty() do
    return 0
  end if
  return values.sum()
end fn total
```

Method	Result
`len()`, `is_empty()`	length, empty or not
`get(i)`	`Option`: `Some(x)` or `None` outside the list
`first()`, `last()`	`Option`
`list[i]`	the element; **outside the list the program stops**
`contains(x)`	`bool`
`map(f)`, `filter(f)`	new list
`sum()`	the sum; stops on an empty list, so check `is_empty()` first
`sorted()`, `reversed()`, `slice(start, end)`	new list
`join(sep)`	joins a `List[Str]`
`push(x)`, `pop()`, `clear()`	change the list: only on a `var` or an `edit` parameter; `pop()` returns an `Option`

9. Your own types

```
module shapes

pub type Point do
  x: i64
  y: i64
end type Point

pub type Shape do
  case Circle(radius: i64)
  case Rectangle(width: i64, height: i64)
  case Empty
end type Shape

pub type Percent
  invariant: value >= 0 and value <= 100
do
  value: i64
end type Percent

pub type Cents = i64

fn manhattan(a: Point, b: Point) -> i64
  effects: none
  example: manhattan(Point(x: 0, y: 0), Point(x: 3, y: -4)) == 7
do
  return abs(a.x - b.x) + abs(a.y - b.y)
end fn manhattan

fn area(s: Shape) -> i64
  effects: none
  example: area(Shape.Rectangle(width: 2, height: 3)) == 6
  example: area(Shape.Empty) == 0
do
  match s do
    case Circle(r):
      return 3 * r * r
    case Rectangle(w, h):
      return w * h
    case Empty:
      return 0
  end match
end fn area
```

```
fn apply(price: Cents, discount: Percent) -> Cents
  effects: none
  example: apply(Cents(1000), Percent(value: 10)) == Cents(900)
do
  return price - price * discount.value / 100
end fn apply

fn euros(amount: Cents) -> i64
  effects: none
  example: euros(Cents(1250)) == 12
do
  let Cents(c) = amount
  return c / 100
end fn euros
```

- **Structures** list their fields; build them by name, `Point(x: 1, y: 2)`, or by position, `Point(1, 2)`; read a field with `p.x`.
- **Sum types** list their cases with `case`, with or without fields; build them with `Shape.Empty` or `Shape.Rectangle(width: 2, height: 3)`.
- **Invariants** go before `do` and are checked every time a value is built: `Percent(value: 120)` stops the program.
- **Distinct types**: `type Cents = i64` is *not* a synonym. A `Cents` and an `i64` do not mix by mistake: `Cents(5)` builds one, `let Cents(c) = amount` takes the number out. Between two `Cents` the usual arithmetic works, and a `Cents` can be multiplied or divided by a plain number.

10. match

```
module matching

type Command do
  case Move(steps: i64)
  case Turn(degrees: i64)
  case Stop
end type Command

fn describe(c: Command) -> Str
  effects: none
  example: describe(Command.Move(steps: 0)) == "stay"
  example: describe(Command.Move(steps: 3)) == "move 3"
  example: describe(Command.Stop) == "stop"
do
  match c do
    case Move(n) if n == 0:
      return "stay"
    case Move(n):
      return "move {n}"
    case Turn(d):
      return "turn {d}"
    case Stop:
      return "stop"
  end match
end fn describe

fn sign_name(n: i64) -> Str
  effects: none
  example: sign_name(0) == "zero"
  example: sign_name(5) == "other"
do
  match n do
    case 0:
      return "zero"
    case _:
      return "other"
  end match
end fn sign_name
```

- `case Pattern:` then the statements on the following lines. Patterns: a variant with its fields (`Move(n)`), a literal (`0`, `"yes"`), a tuple (`(a, b)`), a name that takes any value, or `_`.
- A guard adds a condition: `case Move(n) if n == 0:`.

- **A match must cover every case**: forgetting a variant is an error that names the missing one. Add the case or a final `case _:`.
- To test a single shape without a full match: `value is Some(x)` gives a `bool`.

11. Errors as values: Option and Result

Tessera has no exceptions and no `null`. A function that may fail says so in its return type.

```
module parsing

pub type AgeError do
  case Empty
  case NotANumber
  case TooOld(age: i64)
end type AgeError

fn parse_age(text: Str) -> Result[i64, AgeError]
  effects: none
  example: parse_age("42") == Ok(42)
  example: parse_age("") == Err(AgeError.Empty)
  example: parse_age("4x") == Err(AgeError.NotANumber)
  example: parse_age("200") == Err(AgeError.TooOld(age: 200))
do
  if text.is_empty() do
    return Err(AgeError.Empty)
  end if
  if not text.is_digits() do
    return Err(AgeError.NotANumber)
  end if
  let age = text.to_int().unwrap_or(0)
  if age > 150 do
    return Err(AgeError.TooOld(age: age))
  end if
  return Ok(age)
end fn parse_age

fn age_gap(a: Str, b: Str) -> Result[i64, AgeError]
  effects: none
  example: age_gap("30", "42") == Ok(12)
  example: age_gap("30", "") == Err(AgeError.Empty)
do
  let x = parse_age(a)?
  let y = parse_age(b)?
  return Ok(abs(x - y))
end fn age_gap

fn first_even(values: List[i64]) -> Option[i64]
  effects: none
  example: first_even([1, 3, 4, 6]) == Some(4)
  example: first_even([1, 3]) == None
do
  for v in values do
    if v % 2 == 0 do
```

```
      return Some(v)
    end if
  end for
  return None
end fn first_even
```

- `r?` gives the value inside `Ok` or `Some`, or returns the `Err` or `None` straight away. It works only inside a function that returns the same kind (`Result` or `Option`).
- `unwrap_or(default)` gives the value, or `default` on `Err` or `None`.
- `is_ok()`, `is_err()`, `is_some()`, `is_none()` test the kind.
- **A `Result` cannot be ignored**: a statement that produces a `Result` without using it is an error. Handle it with `?`, `match` or `unwrap_or`.

12. Contracts and examples

Contracts turn assumptions into checks:

- `requires:` is checked when the function is called: if it is false, the *caller* made a mistake.
- `ensures:` is checked when the function returns; `result` is the returned value. With `implies` a condition applies only in some cases: `ensures: result is Ok(c) implies c >= 0` binds `c` for the right-hand side.
- `invariant:` (section 9) is checked when a value of the type is built.
- `example:` clauses are tests that live in the signature: `python -m tessera test file.tsr` runs them all.

A violated contract stops the program with a message that names the function and the condition. examples/prices.tsr shows all of them together.

13. Effects and processes

Any function that does something besides computing must declare it in `effects:`. The prototype knows two effects:

Effect	Built-in function
`console`	`print(value)`
`fs`	`read_text(path)`, which returns `Result[Str, Str]`

A function with `effects: none` cannot call `print`, nor any function that declares an effect: the error says which effect to add, and where. That is how you know, from the signature alone, that a function touches neither files nor the screen.

The program starts from a `process` (or from a `fn main()`). A process also says which **permissions** it receives:

```
process main
  effects: console, fs
  grants: fs.read("data")
do
  match read_text("data/welcome.txt") do
    case Ok(text):
      print("Read: {text.trim()}")
    case Err(problem):
      print("Error: {problem}")
  end match
end process main
```

- `grants: fs.read("data")` allows reading only under the `data` folder; any other path is refused. Outside a process there are no file permissions at all.
- Relative paths start from the folder of the main file, not from where you launch the command.
- `platform: ecc, spectre_mitigations` lists what the machine must guarantee: the *hardware sentinel* checks it before starting, and refuses to start if a requirement cannot be verified.
- `limits: memory 512MB` is read but not enforced yet.

14. Ownership: edit and take

Parameters are read-only. Two words, repeated at the call so you can see what changes:

```
module ownership

fn add_tax(edit prices: List[i64], percent: i64)
  effects: none
do
  var i = 0
  while i < prices.len() do
    prices[i] = prices[i] + prices[i] * percent / 100
    i += 1
  end while
end fn add_tax

fn close(take items: List[Str]) -> Str
  effects: none
  example: close(["a", "b"]) == "a, b"
do
  return items.join(", ")
end fn close

fn main()
  effects: console
do
  var prices = [100, 250]
  add_tax(edit prices, 10)
  print(prices)
  var names = ["bolts", "screws"]
  print(close(take names))
end fn main
```

- `edit` works on the caller's variable, which must be a `var`.
- `take` consumes it: after `close(take names)`, using `names` again is an error. A temporary value (a list written on the spot, the result of a call) can be passed to a `take` parameter without writing `take`.
- The same variable cannot appear twice in one call if one of the two is `edit` or `take`.
- `take` of an outer variable inside a loop is an error: on the next round the value would be gone.
- There are no lifetimes and no references inside structures: that is the whole model.

15. Secret and Untrusted

```
module security

fn check_pin(pin: Secret[Str], attempt: Str) -> bool
  effects: none
  example: check_pin(Secret("1234"), "1234")
  example: not check_pin(Secret("1234"), "0000")
do
  return pin == Secret(attempt)
end fn check_pin

fn main()
  effects: console
do
  let pin = Secret("1234")
  print(check_pin(pin, "0000"))
  let input = Untrusted("42")
  match input.validate(fn(s) => s.is_digits()) do
    case Ok(text):
      print("valid: {text}")
    case Err(problem):
      print("rejected: {problem}")
  end match
end fn main
```

- A `Secret` can be passed around and compared with another `Secret`, but its content never comes out: **printing or interpolating it is an error**, so a password cannot end up in a log by mistake.
- `Untrusted` marks data from outside (a user, a network, the output of a language model). The only way to use what is inside is `validate(check)`, which returns `Ok` only if the check passes.

16. Compiling to WebAssembly

```
python -m tessera build examples/numbers.tsr -o build
```

```
python -m tessera call examples/numbers.tsr gcd 48 18 --wasm
```

The compiler handles functions with `effects: none` that use only `i32`, `i64`, `f64` and `bool`: variables, `if`, `while`, `for` over ranges, calls between compiled functions, `to_f64`, `to_int`, `abs` and contracts. Other functions are skipped, and the compiler says why. Compiled code keeps the same guarantees: overflow, a violated contract or a division by zero stop it with a trap, exactly as the interpreter stops. The tests compare the two on random arguments.

17. Error messages

Every error has a kind, a line and, where possible, a hint:

Kind	Examples
syntax	a block closed out of order, `let mut`, `&&`, a clause in the wrong place
type	ints and decimals mixed, a `Result` ignored, a `match` that misses a case
effects	`print` in a function with `effects: none`
ownership	changing a `let`, using a value after `take`
contract	a `requires`, `ensures` or `invariant` that is false
execution	overflow, division by zero, index out of range
security	printing a `Secret`, `unsafe` outside a kernel module

With `--json` the errors come one per line, in a form that programs (and models) can read. Syntax errors are in English, with a precise hint for habits from other languages. **In the 0.2 prototype the other messages are still in Italian**; they will be translated.

18. Not available yet

The paper describes more than the prototype implements. Not available in 0.2:

- static type checking (types are checked while running, at function boundaries);
- interfaces for generic functions, maps, network, `llm`, tensors and differentiation;
- formal proofs, external functions (`extern`), packages;
- enforcement of `limits` and the IPv6 identity of processes (read, not applied).

`unsafe` blocks exist only in modules with `profile: kernel`.

19. Coming from another language

Wrong	Right
`fn f(x: i64) -> i64 {`	signature, then `effects: none` on its own line, then `do`
`let mut x = 0`	`var x = 0`
`if x > 0 then` / `if x > 0:`	`if x > 0 do` … `end if`
`elif` / `else do`	`else if x > 0 do` / `else` alone
`end` alone	`end if`, `end for`, `end while`, `end match`, `end fn name`
`&&`, `||`, `!x`	`and`, `or`, `not x`
`0 <= x < n`	`0 <= x and x < n`
`1..=n`	`1..n + 1`
`|x| x * 2`, `lambda x: x * 2`	`fn(x) => x * 2`
`x++`, `;`	`x += 1`, nothing
`"a" + "b"`	`"{a}{b}"`
`'c'`	`"c"`
`str`, `int`, `Vec<i64>`, `String`	`Str`, `i64`, `List[i64]`
`case Ok(n) => ...`	`case Ok(n):` and the statements on the next lines
`len(x)`, `x.len`	`x.len()`
`null`, exceptions	`Option`, `Result`
`# comment`, `// comment`	`-- comment`

Tessera is created by **Stefano Stegani - Airgrid Network - www.airgrid.it**. Apache 2.0 licence, with the attribution notice in NOTICE. DOI 10.5281/zenodo.23855820.